COMP3023: Design and Analysis of Algorithms

Lecture 1: Introduction

Computer Science and Technology
United International College

Outline

Contact for Section 1001

- **Instructor:** Dr. Jiaxing Chen
  - Office: T7-102-R16
  - Email: jiaxingchen@uic.edu.cn
- **TA:** Mr. Shichen Zhang
  - Office: T3-502-R26-H6
  - Email: shichenzhang@uic.edu.cn
  - Please make an appointment in advance.
- **Lecture Schedule:**
  Mon. 10:00 - 11:50 at T29-303
  Thu. 15:00 - 15:50 at T29-405
- **Tutorial Schedule:** TBD

Contact for Section 1002 & 1005

- **Instructor:** Dr. Zhiyuan Li
  - Office: T3-502-R6
  - Email: goliathli@uic.edu.cn
- **TA:** Ms. Cici Chen
  - Office: T3-602-R25-H8
  - Email: chongchen@uic.edu.cn
  - Please make an appointment in advance.
- **Lecture Schedule:**
  Tue. 14:00 - 15:50 at T7-503
  Thu. 16:00 - 16:50 at T7-304
- **Tutorial Schedule:** TBD

Contact for Section 1003

- **Instructor:** Dr. Wentao Cheng
  - Office: T7-102-R13
  - Email: wentaocheng@uic.edu.cn
- **TA:** Ms. Cici Chen
  - Office: T3-602-R25-H8
  - Email: chongchen@uic.edu.cn
  - Please make an appointment in advance.
- **Lecture Schedule:**
  Mon. 15:00 - 16:50 at T7-503
  Wed. 13:00 - 13:50 at T6-503
- **Tutorial Schedule:** TBD

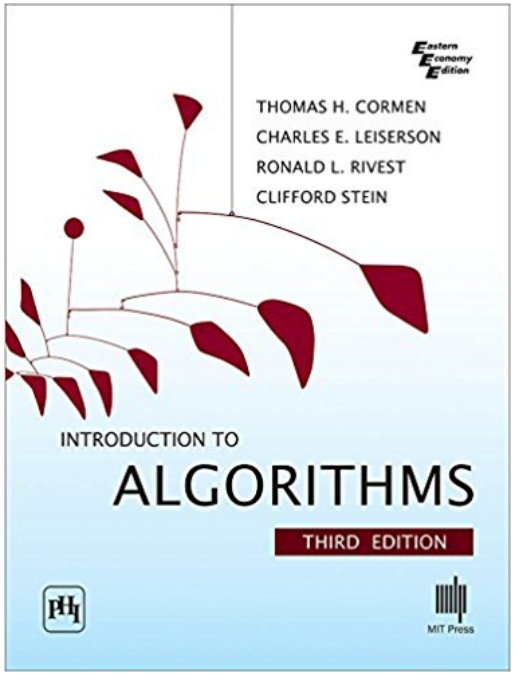
Contact for Section 1004

- **Instructor:** Dr. Zhiyuan Li
  - Office: T3-502-R6
  - Email: goliathli@uic.edu.cn
- **TA:** Ms. Rubin Zhou
  - Office: T3-502-R26-H25
  - Email: rubinzhou@uic.edu.cn
  - Please make an appointment in advance.
- **Lecture Schedule:**
  Tue. 10:00 - 11:50 at T6-503
  Thu. 15:00 - 15:50 at T6-503
- **Tutorial Schedule:** TBD

Textbook

Introduction to Algorithms, Third Edition

By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein



References

- Programming Pearls (2nd Edition), by Jon BENTLEY. Addison-Wesley,2000.
- Computers and intractability: A guide to the theory of NP-completeness, by Michael R. GAREY and David S. JOHNSON. W.H. Freeman, 1979.
- Algorithms in C++ (3rd Edition) Volumes 1 and 2, by Robert Sedgewick. Addison-Wesley, 1998.
- Online Programming Learning Platform
  - Leetcode, https://leetcode.com/problemset/algorithms
  - HackerRank, https://www.hackerrank.com
  - LintCode,https://www.lintcode.com

Syllabus

- Introduction
- Divide and Conquer
  - Maximum Contiguous Subarray Problem
  - Polynomial Multiplication Problem
  - Randomized Partition and Randomized Selection
  - Deterministic Linear-Time Selection
- Graph Algorithms
  - DFS Search and It's Applications
  - Minimum Spanning Trees
  - Dijkstra's Shortest Path

Syllabus

- Dynamic Programming
  - The Knapsack Problem
  - Chain Matrix Multiplication
  - All Pairs Shortest Path Problem
- Greedy Algorithm
  - The Fractional Knapsack
  - Huffman Coding
- NP and NP-completeness (Optional)

Course Assessment

Grading:

- Written Assignments (15%)
- Programming Assignments (15%)
- Midterm Test (30%)
- Final Exam (40%)
- No late submission accepted
- Written assignments in PDF format
- Final exam below 20/100 - Failed directly
- Final exam below 30/100 - Down grade on letter grade

Zero Tolerance of Plagiarism

- Plagiarism in exams - Fail the course and reported to AR
- Plagiarism in assignments
  - First time - Get 0 in the assignment and a warning
  - Second time - Fail the course and reported to AR
- Both coping and copied are plagiarism. We are not detectives. We do not investigate.
- What if your best friend ask you for your assignment?
- You cannot show your answer directly.
- You can give your friend some hints.
- Cheating students will be on a blacklist, and will be taken a special care even in other courses.

LLM Policy

LLMs are not avoidable in teaching and learning. Thus, students should follow these.

- The principle is "**LLM is a 24-hour teacher**".
- Teachers can answer your questions.
- Teachers cannot do assignments for you.
- Students cannot use LLMs in exams.

For example, if you want to know how to do 17×23

- you can ask your teacher "how to calculate multiplication?"
- you cannot ask your teacher "what is 17×23?"

Asking LLM "what is 17×23?" is same as cheating.
This is **not allowed**.

LLM Policy

Student: I think students should learn how to use AI.

Goliath: You indeed should learn how to use a calculator. But, it does not mean that you don't need to learn how to calculate arithmetic by yourself.

Student: If AI can answer questions correctly, why do I need to do it by myself?

Goliath: AI is a machine made by human. Machines do not make mistakes but human do. If it happens, how do you verify and correct?

Student: As long as AI can do most of the questions correct, I am happy. I don't care the rest.

Goliath: Change you major to other programs. They don't care either. **Mistakes in engineering kill people.** Don't be a murderer.

Computational Problems

Definition (Computational Problem)

A computational problem is a specification of the desired input-output relationship.

- In mathematics, a relationship is expressed as a relation from input set *INPUT* to output set *OUTPUT*.
- So, a problem is a subset of $INPUT \times OUTPUT$ (cartesian product).

Definition (Instance)

An instance of a problem is an object in *INPUT*, which contains all information needed to compute a solution to the problem.

Example

To formally define a computational problem, we need to define the type of input and output. For example,

Example (Sorting in ascending order)

Computational problem **SORTING** is defined as

- **Input:** A sequence A of n numbers, denoted as $A = \langle a_1, a_2, \cdots, a_n \rangle$, for $n \geq 1$.
- **Output:** A permutation (reordering) of A, denoted as $A' = \langle a'_1, a'_2, \cdots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

- $\langle a_1, a_2, \cdots, a_n \rangle$ defines the type of input and output, which is a sequence.
- The inequalities $a'_1 \leq a'_2 \leq \cdots \leq a'_n$ after the phrase "such that" describes the desired condition on the output.
- Some problems are described by "Question" instead of "Output".

Example

For the sorting problem,

- *INPUT* (the set of all inputs or instances) is the set of all sequences;
- same for *OUTPUT*;
- so, the problem is a relation from sequences to sequences;
- a possible instance can be $\langle 8, 3, 6, 7, 1, 2, 9 \rangle$;
- the desired output is $\langle 1, 2, 3, 6, 7, 8, 9 \rangle$.

Algorithms

Definition (Algorithm)

An algorithm is a **well defined** computational procedure that transforms inputs into outputs, **achieving the desired input-output relationship**.

- **procedure**: An algorithm is a sequence of instructions.
- **well defined**: Each instruction has no ambiguous.
- **computational**: The number of instruction is finite and each instruction is executable by a computer.
- **achieve the relationship**: The algorithm is correct.

Definition (Algorithm Correctness)

A correct algorithm halts with the correct output for every input instance. We can then say that the algorithm solves the problem.

An incorrect algorithm might

- not halt at all on some input instances,
  - for example a dead loop;
- halt with an incorrect answer for some instances,
  - for example $\langle 2, 3, 7, 9, 8, 6, 1 \rangle$.

Algorithms

An algorithm can be expressed on different levels.

- The **low level** presentation of an algorithm is a piece of source code in a specific language, like C or Java.
  An algorithm expressed on this level can be directly executed by a computer.
- The **high level** presentation of an algoirthm is some instructions described in a human language, like English or Chinese.
  These instructions can be easily understood by human.
- The **intermediate level** presentation of an algorithm is a piece of **pseudo code**, which is in between the high level and the low level.
  Human can understand the meaning, but not easy. And the intermediate presentation can be implemented by any programming language.

Algorithms

- We consider that a lower level is closer to machines, while a higher level is more natural to human.
- In this course, we focus on the intermediate level (pseudo code) and sometimes talk about the high level.
- There are some other levels which may be higher or lower, but we do not discuss them.

Algoritm Design

Remember that this couse is "**Design** and **Analysis** of **Algorithm**", but not simply **Algorithm**. We need to know how algorithms are designed.

- To design an algorithm for a given problem, there is **no** procudure can **always** produce a correct algorithm in general. (The reason is explained in COMP4003.)
- However, we can follow this scheme by assuming you (the designer) are the solver of the problem.
  1. Try to fully understand the problem. For an arbitrary input, you should find the correct output.
  2. Reason the method you used to find the output in step 1.
  3. Formally present your method in a natural language (high level presentation). The description must be universal (works for all instances).
  4. Verify the correctness of your algorithm by 1) a formal mathematical proof, or 2) a large amount of test cases.
  5. Convert the high level presentation in a piece of pseudo code.

Pseudo Code

To write elegant pseudo codes, we introduce the syntax.

- An algorithm is bounded by two horizontal bars.
- An algorithm has a number, a name, and parameters, like declaring a function.
- Another horizontal bar below the algorithm declaration.
- Define the input and the output before instructions.
- Each line of pseudo codes has a number.
- The instructions can be
  - assignment "=",
  - math formulas,
  - function calls to existing algorithms.

Pseudo Code

- Sometimes instructions can also simple computations like $SWAP(a, b)$, MIN, MAX, etc.
- This presentation is slightly higher than pseudo codes and improves the readability.
- However, you have to make sure each of such instructions has no ambiguous and executable by a computer (the property of an algorithm).

Pseudo Code

For flow controls,

- every scope starts from one of
  - **begin** (the start of a piece of pseudo codes),
  - **for ... do**,
  - **while ... do**, or
  - **if ... then**;
- and ends by **end** (like tags in HTML).
- Sometimes we can also use vertical bars to express scopes and omitting **end** to save some space.
- **for** needs an iterator, like
  - **all** i in a set;
  - $i = 0$ to 10.
- **while** and **if** need a predicate, like $i > 0$.
- An **if** can be paired with any number of **else if** and at most one **else**.

Example of Algorithm Design: Sorting

Given an array of numbers, we want to rearrange the numbers in the non-decreasing order.

Step 1: Understand the problem.

List some instances

- Input: $\langle 5, 2, 4, 6, 1, 3 \rangle$ Output: $\langle 1, 2, 3, 4, 5, 6 \rangle$
- Input: $\langle 3, 5, 7, 1, 7, 9, 2 \rangle$ Output: $\langle 1, 2, 3, 5, 7, 7, 9 \rangle$
- ...

In general, the problem is defined as

Definition (Sorting)

- **Input:** An array A of n integers (or numbers)
- **Output:** A permutation of A such that $A[i] \leq A[j]$ for all $1 \leq i \leq j \leq n$.

Example of Algorithm Design: Sorting

Given an array of numbers, we want to rearrange the numbers in the non-decreasing order.

Step 2: How do you find a solution? (For example $\langle 5, 2, 4, 6, 1, 3 \rangle$)

1. Pick the first number 5, insert it into $\langle\rangle$ (empty array), and get $\langle 5 \rangle$.
2. Pick the second number 2, insert it into $\langle 5 \rangle$ (the array from step 1), and get $\langle 2, 5 \rangle$.
3. Pick the third number 4, insert it into $\langle 2, 5 \rangle$, and get $\langle 2, 4, 5 \rangle$.
4. **And so on.** (Goliath is just lazy and does not want to write every step.)

Example of Algorithm Design: Sorting

Given an array of numbers, we want to rearrange the numbers in the non-decreasing order.

Step 3: High-level presentation

We need to specify some details and generalize the above method.

- What do we get after i-th iteration?
  - A partially sorted array, $A[1 \cdots i]$.
- How do you understand "**And so on**"?
  - In i-th iteration, insert the i-th number into the sorted array $A[1 \cdots i-1]$.
- What is the rage of i?
  - $2 \leq i \leq A.length$ (The first number is omitted.)

Example of Algorithm Design: Sorting

Given an array of numbers, we want to rearrange the numbers in the non-decreasing order.

Step 3: High-level presentation

After combining everything, the high-level presentation can be

1. For $2 \leq i \leq A.length$,
2. **Insert** $A[i]$ into $A[1 \cdots i-1]$
3. where $A[1 \cdots i-1]$ is the sorted subarray.

- Someone may think this presentation is not high-level enough because it uses some notations like i and A.
- But we need to find a balance between simplicity and accuracy.

Example of Algorithm Design: Sorting

Given an array of numbers, we want to rearrange the numbers in the non-decreasing order.

Step 4: Verify the correctness

- A formal correctness proof is not easy, but it is solid. We will see some proofs in this course.
- Using test cases is relatively easy, but the test cases should cover all situations.
- For your exercises, using test cases is usually enough.

Example of Algorithm Design: Sorting

Given an array of numbers, we want to rearrange the numbers in the non-decreasing order.

Step 5: Pseudo code

We only need to translate the method after step 4 in the syntax of pseudo codes. How to "**insert**" must be specified.

Algorithm 1: INSERTION-SORT(A)

Input: An array A of n integers

Output: The permutation of A such that $A[i] \leq A[j]$ for all $1 \leq i \leq j \leq n$

```
1 begin
2     for j = 2 to A.length do
3         key = A[j]
4         i = j − 1
5         while i > 0 and A[i] > key do
6             A[i + 1] = A[i]
7             i = i − 1
8         A[i + 1] = key
9     return A
```

Algorithm Analysis

- Predict resource utilization
  - Memory (space complexity)
  - Communication bandwidth
  - Computational/Running time (time complexity)
  - Remark: Depends on the model of computation (sequential or parallel). We usually assume sequential computation model.

Generally, by analyzing several candidate algorithms for a problem, we can identify the most efficient one.

Algorithm Analysis

In general, the time taken by an algorithm grows with the size of the input, so it is traditional to describe the running time of a program as a function of the size of its input.

- **Input size**
  - Sorting: number of items in the input.
  - Multiplication: total number of bits needed to represent the input in binary notation.
  - Graph: number of vertices and edges.
- **Running Time**
  - Number of primitive operations used to solve the problem.
  - Primitive operations: addition, multiplication, comparison, assignment, return etc.

Algorithm Analysis

To analyze the running time,

- Write the number primitive operations on each line.
- Count the number rounds of each loop.
- Substitute the time cost for each function call.
- Addup everything to obtian the time cost function.
- For recursions, solve the **recurrence relation**.

Algorithm Analysis Example

Algorithm 2: INSERTION-SORT(A)

Input: An array A of n integers

Output: The permutation of A such that $A[i] \leq A[j]$ for all $1 \leq i \leq j \leq n$

```
1 begin
                                          // 1 initializing j
2     for j = 2 to A.length do            // n − 1 times
                                          //   3 incrementing j
3         key = A[j]                      //   1
4         i = j − 1                       //   2
5         while i > 0 and A[i] > key do   //   1 ∼ (j − 1) times
                                          //     3 guard checking
6             A[i + 1] = A[i]             //     2
7             i = i − 1                   //     2
8         A[i + 1] = key                  //   2
9     return A                            // 1
```

Note: The number of rounds for the "while" loop on line 5 is from 1 to $j - 1$, which depends on the value of $A[i]$.

Three Cases of Analysis

Best Case

- Constraints on the input, other than size, resulting in the fastest possible running time.
- For example, in INSERTION-SORT, the best case occurs if the array is already sorted.
- The "while" loop from line 5 to line 7 is not executed at all in each round of the "for" loop.
- The time cost is
  - Round 1: $3+1+2+3+2$
  - Round 2: $3+1+2+3+2$
  - $\vdots$
  - Round $n-1$: $3+1+2+3+2$
- In total, $T(n) \geq 1+(3+1+2+3+2)(n-1)+1 = 11n-9$.
- Inequality is used here because this is the best case.

Three Cases of Analysis

Worst Case

- Constraints on the input, other than size, resulting in the slowest possible running time.
- For example, in INSERTION-SORT, the worst case occurs if the array is reverse sorted order.
- "while" loop on line 5-7 is executed $j-1$ times in each round.
- The time cost is
  - Round 1: $3+1+2+(3+2+2)1+2$
  - Round 2: $3+1+2+(3+2+2)2+2$
  - $\vdots$
  - Round $n-1$: $3+1+2+(3+2+2)(n-1)+2$
- In total,

$$\begin{aligned} T(n) &\leq 1+6(n-1)+7(1+2+\cdots+n-1)+1 \\ &= 1+6n-6+(7/2)n(n-1)+1 \end{aligned}$$

In this course, we shall usually concentrate on finding only the worst-case running time.

Three Cases of Analysis

Average Case

- Average running time over every possible type of input (usually involve probabilities of different types of input).
- Assuming that each of the $n!$ instances are equally likely.
- The time cost is of the **same order** of the number of integer comparisons.
- On average, we'd expect each element is less than half the elements to its left, the number of comparisons needed is **about** $T(n) \approx \sum_{j=2}^{n} \frac{j-1}{2} = \frac{n(n-1)}{4}$.
- The meaning of "**same order**", "**about**", and "$\approx$" is explained in the following slides.

Growth of Functions

- The order of growth (rate of growth) of the running time of an algorithm gives a simple characterization of the algorithm's efficiency.
- For large inputs, the multiplicative constants and lower-order terms of an exact running time are dominated by the effects of leading term (e.g., an^2 of $an^2 + bn + c$).
- Asymptotic Analysis of algorithms
  - Concern how the running time of an algorithm increases with the size of input *in the limit*, as the size of the input increases without bound.
  - Asymptotic notation: Θ-notation, $\mathcal{O}$-notation. Ω-notation,

Asymptotic Notation- $\mathcal{O}$-notation

- For a given function $g(n)$, we denote $\mathcal{O}(g(n))$ the set of functions
  $\mathcal{O}(g(n)) = \{f(n) : \exists c > 0, n_0, \forall n \geq n_0 (0 \leq f(n) \leq cg(n))\}$.
- $g(n)$ is an asymptotic upper bound for $f(n)$.
- Remark: $f(n) = \mathcal{O}(g(n))$ means that $f(n)$ is a member of set $\mathcal{O}(g(n))$.
- Examples
  - $n^2 + 2n = \mathcal{O}(n^2)$
  - $n \lg n = \mathcal{O}(n^2)$
  - $n^2 \lg n \neq \mathcal{O}(n^2)$
  - $\forall a, b > 1$, $\log_a n = \mathcal{O}(\log_b n)$

Asymptotic Notation- Ω-notation

- For a given function $g(n)$, we denote $\Omega(g(n))$ the set of functions
  $\Omega(g(n)) = \{f(n) : \exists c > 0, n_0, \forall n \geq n_0 (0 \leq cg(n) \leq f(n))\}$.
- $g(n)$ is an asymptotic lower bound for $f(n)$.
- Remark: $f(n) = \Omega(g(n))$ means that $f(n)$ is a member of set $\Omega(g(n))$.
- Examples
  - $n^2 + 2n = \Omega(n^2) = \Omega(n) = \Omega(1)$
  - $n^2 \neq \Omega(n^2 \lg n)$
  - Does $f(n) = \mathcal{O}(g(n))$ imply $g(n) = \Omega(f(n))$?
  - Does $f(n) = \Omega(g(n))$ imply $g(n) = \mathcal{O}(f(n))$?

Asymptotic Notation- Θ-notation

- For a given function $g(n)$, we denote $\Theta(g(n))$ the set of functions
  $\Theta(g(n)) = \{f(n) : \exists c_1 > 0, c_2 > 0, n_0, \forall n \geq n_0 (0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n))\}$.
- $g(n)$ is an asymptotic tight bound for $f(n)$.
- Remark: $f(n) = \Theta(g(n))$ means that $f(n)$ is a member of set $\Theta(g(n))$.
- Examples
  - $n^2 + 2n = \Theta(n^2)$
  - $5n^2 - 2n + 5 = \Theta(n^2 + \lg n)$
- Note that $f(n) = \Theta(g(n))$ if and only if $f(n) = \Omega(g(n))$ and $f(n) = \mathcal{O}(g(n))$

Asymptotic Analysis Example

We analyze the asymptotic time cost of "INSERTION-SORT".

- Best case: $T(n) = 3n - 3 = \Theta(n)$.
- Worst case: $T(n) = 3n - 3 + n(n-1) = \Theta(n^2)$.
- Average case: $T(n) = \frac{n(n-1)}{4} = \Theta(n^2)$.

Machine Dependency

Here is an argument happened few years ago ...

Alice: If I change the code on Line 7 to "$i--$", this line contains only **one** primitive operation. Then, the time cost will be different.

Bob: It does not matter. I have checked GCC compiler. Both "$i--$" and "$i = i - 1$" are translated into the same piece of assembly code, which does subtraction first and assignment second. Thus, "$i--$" also contains two primitive operations.

Alice: This is only one compiler for one specific language. How about JVM for java? If you have not checked all compilers for all langauges, you cannot make such a naïve claim.

Machine Dependency

- The time cost function is of course depends on the implementing language and the executing machines, which is called "machine dependency".
- However, this is course is about algorithms, which should be machine independent. Those difference in time cost functions are minor and should be ignored.
- Asymptotic analysis is used for this goal.
- Both "$i--$" and "$i = i - 1$" takes $\mathcal{O}(1)$ time, irrelevant to the input size.
- The time cost (worst case) after applying asymptotoic analysis is on the next page.

Machine Independent

Algorithm 3: INSERTION-SORT(A)

Input: An array A of n integers

Output: The permutation of A such that $A[i] \leq A[j]$ for all $1 \leq i \leq j \leq n$

```
1 begin
2     for j = 2 to A.length do                  // O(n) times
3         key = A[j]                            //    O(1)
4         i = j − 1                             //    O(1)
5         while i > 0 and A[i] > key do         //    O(j) times at worst
                                                //       O(1)
6             A[i + 1] = A[i]                   //       O(1)
7             i = i − 1                         //       O(1)
8         A[i + 1] = key                        //    O(1)
9     return A                                  // O(1)
```

Note: In this course, we write $\mathcal{O}(1)$ instead of constants 2 or 3 to avoid detailed discussions.

Some thoughts on Algorithm Design

- Algorithm design, as taught in this class, is mainly about designing algorithms that have small $\mathcal{O}()$ running times.
- With "all other things being equal", $\mathcal{O}(n \lg n)$ algorithms will run more quickly than $\mathcal{O}(n^2)$ ones and $\mathcal{O}(n)$ algorithms will beat $\mathcal{O}(n \lg n)$ ones.
- Being able to do good algorithm design lets you identify the hard parts of your program and deal with them effectively.
- Too often, programmers try to solve problems using brute force techniques and end up with slow complicated code. A few hours of abstract thought devoted to algorithm design could have speed up the solutions substantially.

Final Notes

- After algorithm design one can continue on to Algorithm Tuning which would further concentrate on improving algorithms by cutting cut down on the constants in the $\mathcal{O}()$ bounds. This needs a good understanding of both algorithm design principles and efficient use of data structures.
- In this course we will not go further into algorithm tuning.
- For a good introduction, see chapter 9 in Programming Pearls, 2nd Edition by Jon Bentley.

More Tips

- Most of the topics of this course is problem-solver oriented.
- For each problem, you need to understand
  - the problem description,
  - modeling,
  - inputs and expected outcomes.
- For each algorithm, you need to know
  - the problem solved by the algorithm,
  - high-level description of the algorithm (usually the strategy of each algorithm can be described in few sentences),
  - and the time complexity.